IB CS B3 Object-Oriented Programming Revision Notes

First Drafted by TraeWork and edited by R. Guo with the help of Gemini

Guiding question: Is object-oriented programming an appropriate paradigm for solving complex problems?

Scope of this document: Only the content named in B3 is covered.

Language used: All code examples are written in Python 3.

● B3.1 Fundamentals of OOP for a single class

●.1 Evaluate the fundamentals of OOP

Syllabus point: B3.1.1

Object-oriented programming organizes a program around objects. An object combines state, usually stored in attributes, with behaviour, usually defined in methods. A class acts as the blueprint from which many similar objects can be created.

The key OOP ideas named in the syllabus are classes, objects, encapsulation, inheritance, and polymorphism. Even though inheritance and polymorphism are explored more fully later, students should already understand them as central OOP ideas when evaluating why OOP is useful.

●.1.1 Key OOP Concepts

- **Class**: A class is a blueprint, template, or recipe used to create objects. It defines the structure (what attributes it will have) and the behavior (what methods it can run), but it does not contain real data on its own. Writing a class is like drawing the architectural blueprint for a house—you cannot live inside the blueprint itself, but you use it to build actual houses.
- **Object**: An object is a concrete instance of a class created from its blueprint. It is a real, physical entity that occupies space in the computer's memory and holds actual data. If the Class is the blueprint of a house, the Object is the physical house built on a specific street. From one single class blueprint, you can instantiate as many unique objects as you need (e.g., user1 for "Alice" and user2 for "Bob" are both objects of the User class). An object is an abstract entity that consists of **state (attributes)** and related **behavior (methods)**. An object stores its state in fields and exposes its behavior through methods.
- **Attribute**: Attributes of a class in Python are the data stored inside a class or its objects. They represent the state of either the class itself or each instance created from it. There are two types of attributes - **instance attributes** and **class attributes.** Instance attributes belong to each object. They are accessed through **self**. Class attributes belong to the class itself, shared by all instances.

- **Method**: A method is a collection of code that can be identified by its name and accessed at any point in a program by simply using its name. Additionally, methods can accept input values, known as parameters. **Instance method**: a type of method that can be called on an instance of a class, and has access to both instance variables (which are unique to each instance) and class variables (which are shared across all instances of the class). In Python, it always has self as the first parameter. A **class method** is a method that belongs to the class itself, not to any individual instance. It works with class-level data and can be called without creating an object. In Python, it uses the **@classmethod** decorator and **cls** instead of self.
- **Encapsulation**: Encapsulation is a concept that involves concealing the implementation details of data within a class and providing access to that data from outside the class only through methods such as accessor or mutator methods.
- **Inheritance**: Inheritance models an “is-a” relationship. A class (called the child or subclass) can inherit attributes and methods from another class (called the parent or superclass). Inheritance lets a class **reuse** and **extend** another class’s code.
- **Polymorphism**: Polymorphism means different classes can be used through the same interface, allowing one method name to produce different behaviors depending on the object. It lets subclasses override parent methods so each object responds in its own way. In short, polymorphism enables “one interface, many implementations. There are two types of polymorphism - **method overloading** and **method overriding**.
- **Special Methods:** an **accessor (getter)** returns an attribute of an object. A **mutator (setter)** is a method that updates the value of a private attribute, usually with **validation** or **side-effects** (the state of the object is changed). A **constructor** in Python is the special method **__init__** that runs automatically when you create an object. Its job is to initialize instance variables and set up the object’s initial state.
- **Access Modifiers:** In Python, access modifiers are not enforced by the language the way Java or C++ do. Instead, Python uses naming conventions to signal whether an attribute or method is meant to be public or private. By default attributes and methods are **public**. A variable or a method without a leading underscore is public. It means that they are accessible from anywhere. A double underscore prefix indicates that it is **private**. Python uses name mangling to change its name behind the scene to make it harder to access accidentally.

●.1.2 Why OOP can help

OOP is often a strong choice when a problem involves several real-world entities with different responsibilities. A library system can be broken into Book, Member, and Loan objects; a game can be broken into Player, Enemy, and Item objects; an online shop can be broken into Product, Cart, and Order objects.

By giving each class a clear role, the program becomes easier to reason about. Changes in one part of the system can often be made inside one class instead of being scattered across unrelated code.

●.1.3 Advantages and disadvantages

Advantages:

- Code Reuse: Class created for Object-Oriented programs can easily be reused in other programs/other parts of the program.
- Speed up development as work can be split up across the development groups, which allows developers to work on different parts of the program simultaneously. Therefore, increase the speed of the development.
- Quicker to test: as each module is small and independent and therefore be tested separately.

Disadvantages:

- increased complexity for small problems
- Size: Object-Oriented programs are in general larger than other programs. It may pose a problem when there is only limited storage.
- Effort: Object oriented programs require a lot of work to create. Specifically, a great deal of planning goes into an object oriented program well before a single piece of code is ever written.
- Unsuitable for minor projects. OOP increases complexity but gains little for small projects.

●.1.4 Python example

Python

```python
class LibraryBook:
    def __init__(self, title: str, author: str):
        self._title = title
        self._author = author
        self._on_loan = False

    def borrow(self) -> bool:
        if self._on_loan:
            return False
        self._on_loan = True
        return True

    def return_book(self) -> None:
        self._on_loan = False

book = LibraryBook("1984", "George Orwell")
print(book.borrow())   # True
print(book.on_loan)    # True
```

In this example, the object stores its own state in `_title`, `_author`, and `_on_loan`. These instance variables are considered as private variables. Variables with two leading underscore characters are also private variables. They trigger the name mangling mechanism, making it difficult to access from the outside.

The methods `borrow()` and `return_book()` are the behaviours that control that state.

●.2 Construct a design of classes, their methods and behaviour

Syllabus point: B3.1.2

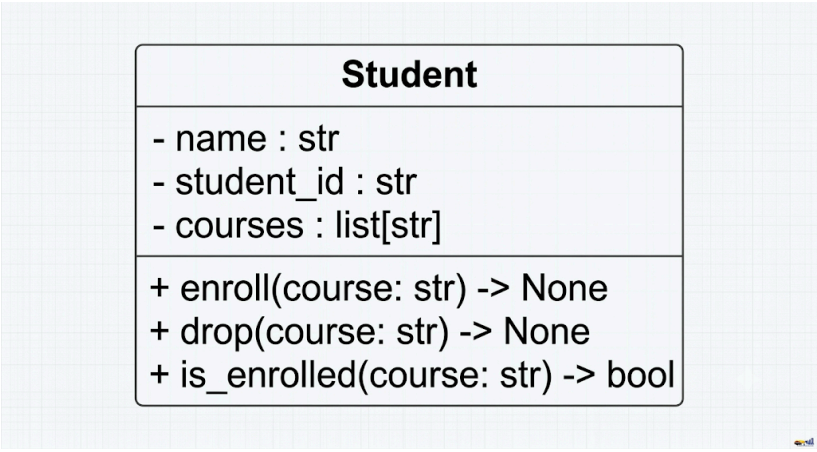
Class design begins before code is written. Start by identifying the important entities in the problem, the data each entity must store, and the actions each entity must perform. A strong design gives each class one clear responsibility.

The syllabus also points to UML class diagrams. A UML class diagram is a planning tool that shows class names, attributes, methods, and relationships between classes. Even if the final code is Python, UML still helps structure the design.

●.2.1 Questions to ask when designing a class

- What real-world or logical entity does this class represent?
- What data must every object of this class store?
- What actions should the object be able to perform?
- Which methods change the object state, and which simply return information?
- Should this responsibility belong in this class, or in some other class?

●.2.2 UML Class Diagram (Single Class)



●.2.3 Python class from the design

```python
class Student:
    def __init__(self, name: str, student_id: str):
        self.name = name
        self.student_id = student_id
        self.courses: list[str] = []

    def enroll(self, course: str) -> None:
        if course not in self.courses:
            self.courses.append(course)

    def drop(self, course: str) -> None:
        if course in self.courses:
            self.courses.remove(course)

    def is_enrolled(self, course: str) -> bool:
        return course in self.courses
```

The design is simple because the class has one main job: manage a student's course list. This is usually better than putting student registration, fee calculation, timetable printing, and messaging into the same class.

●.3 Distinguish between static and non-static variables and methods

Syllabus point: B3.1.3

The syllabus uses the terms static and non-static. Python does not use the keyword `static` for variables, but the idea still exists through class attributes, instance attributes, class methods, and static methods.

●.3.1 Python equivalents

- **Instance attribute**: belongs to one object. Accessed through `self`, for example `self.speed`.
- **Class attribute**: belongs to the class itself and is shared by all objects, for example `Bike.count`.
- **Instance method**: works with one object's data and receives `self`.
- **Class method:** uses `@classmethod`, receives `cls`, and usually works with class-level data.
- **Static method:** uses `@staticmethod` and behaves like a utility function placed inside the class namespace.

Use an instance attribute when each object should keep its own separate value. Use a class attribute when one value should be shared across every object of that class.

●.3.2 Python example

```python
class Bike:
    count = 0  # class attribute

    def __init__(self, speed: int):
        self.speed = speed  # instance attribute
        Bike.count += 1

    def accelerate(self, amount: int) -> None:
        self.speed += amount

    @classmethod
    def total_bikes(cls) -> int:
        return cls.count

    @staticmethod
    def km_to_miles(km: float) -> float:
        return km * 0.621371
```

Every Bike object has its own `speed`, so `speed` is an instance attribute. The class-wide value `count` tracks how many bikes have been created, so it is a class attribute shared across all objects.

●.4 Construct code to define classes and instantiate objects

Syllabus point: B3.1.4

To define a class in Python, use the `class` keyword. To instantiate an object, call the class name like a function. Python treats classes as callable objects, creates a new instance, and then runs `__init__` to initialize the new object's state.

In beginner OOP discussions, `__init__` is often described as a constructor because it is the method that initializes object data. More precisely, Python creates the object first and then calls `__init__` to customize it.

●.4.1 Python example

```python
class Car:
    def __init__(self, make: str, speed: int = 0):
```

```
        self.make = make
        self.speed = speed

    def accelerate(self, amount: int) -> None:
        self.speed += amount

my_car = Car("Toyota")
my_car.accelerate(15)
print(my_car.make)   # Toyota
print(my_car.speed)  # 15
```

●.4.2 Key ideas to explain

- **Class definition**: the blueprint is created.
- **Instantiation** and **Initialization:** Instantiation is the process of allocating memory to create a concrete instance of a class, whereas initialization is the process of setting that newly created instance's initial state (assigning default or custom values to its fields/attributes).
- **Method call**: behaviour is applied to that particular object.

●.5 Common methods

●.5.1 Constructor

A **constructor** in Python is a special method called automatically when a new object of a class is instantiated. Its main role is to initialize the object's instance variables (state) with starting values.

In Python, the `__init__()` magic method serves as the standard constructor.

A class can have multiple constructors with different parameters.

●.5.2 Getters and Setters

Getters and **setters** (also known as accessors and mutators) are public methods used in Object-Oriented Programming (OOP) to safely access and update private class attributes, upholding the core principle of **encapsulation**.

- Getters:**Reads** and retrieves the current value of a private variable.
- Setters: **Writes** or updates the value of a private variable, usually with validation checks.

In Python, getters and setters are most cleanly implemented using the `@property` decorator, which allows you to access methods like regular attributes while maintaining control over data validation.

Python

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        # Calls the setter automatically during initialization
        self.balance = balance

    # Getter for balance
    @property
    def balance(self):
        return self._balance

    # Setter for balance (includes validation)
    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("Balance cannot be negative.")
        self._balance = value


# --- Usage Example ---
account = BankAccount("Alice", 100)

# GETTER: Accessed like an attribute (no parentheses)
print(account.balance)  # Output: 100

# SETTER: Modified using standard assignment syntax
account.balance = 250   # Updates _balance to 250
print(account.balance)  # Output: 250

# SETTER VALIDATION: Triggers ValueError
account.balance = -50   # Raises: ValueError: Balance cannot be negative.
```

In conventional Object-Oriented Programming (similar to Java or C++ style), getters and setters are defined as explicit, standard methods prefixed with `get_` and `set_`.

Python

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        # Using a leading underscore to signal that _balance is protected/internal
        self.set_balance(balance)

    # Conventional Getter
    def get_balance(self):
        return self._balance
```

```
    # Conventional Setter
    def set_balance(self, value):
        if value < 0:
            raise ValueError("Balance cannot be negative.")
        self._balance = value


# --- Usage Example ---
account = BankAccount("Alice", 100)

# GETTER: Explicit method call with parentheses
print(account.get_balance())  # Output: 100

# SETTER: Explicit method call passing the new value
account.set_balance(250)
print(account.get_balance())  # Output: 250

# SETTER VALIDATION: Triggers ValueError
account.set_balance(-50)      # Raises: ValueError: Balance cannot be negative.
```

You must know both ways to define getters and setters.

- 5.3 __str__ and __eq__

Both `__str__` and `__eq__` are special "dunder" (double underscore) methods in Python that customize how your objects behave with built-in functions and operators.

- **`__str__`**: Controls what `print(obj)` or `str(obj)` returns (human-readable string representation).
- **`__eq__`**: Controls how the equality operator `==` evaluates two distinct objects based on their values rather than their memory locations.

Python

Python

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn

    # Defines user-friendly string formatting for print() and str()
    def __str__(self):
```

```
        return f"'{self.title}' by {self.author}"

    # Defines logic for the == operator
    def __eq__(self, other):
        # Good practice: check if the other object is an instance of the same
class
        if isinstance(other, Book):
            return self.isbn == other.isbn
        return False


# --- Creating Objects ---
book1 = Book("1984", "George Orwell", "978-0451524935")
book2 = Book("1984", "George Orwell", "978-0451524935")  # Separate memory
location, same ISBN
book3 = Book("Brave New World", "Aldous Huxley", "978-0060850524")


# --- Testing __str__ ---
# Without __str__, print(book1) would output <__main__.Book object at 0x7f8...>
print(book1)        # Output: '1984' by George Orwell
print(str(book3))   # Output: 'Brave New World' by Aldous Huxley


# --- Testing __eq__ ---
# Without __eq__, book1 == book2 would be False because they are different memory
addresses.
print(book1 == book2)  # Output: True (ISBNs match)
print(book1 == book3)  # Output: False (ISBNs differ)

# Safely handles comparison with non-Book objects
print(book1 == "1984")  # Output: False (returns False via isinstance check
instead of crashing)
```

Both `__str__` and `__eq__` are special "dunder" (double underscore) methods in Python that customize how your objects behave with built-in functions and operators.

- **`__str__`**: Controls what `print(obj)` or `str(obj)` returns (human-readable string representation).
- **`__eq__`**: Controls how the equality operator `==` evaluates two distinct objects based on their values rather than their memory locations.

●.6 Object Reference

Object reference: An object reference in Python is simply a name (or variable) that points to an object stored in memory. The name does not contain the object — it only refers to it.

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount


# --- 1. Object Creation (Instantiation & Reference) ---
# 'acc1' does NOT hold the balance directly; it stores a reference (memory address) to the account instance.
acc1 = BankAccount("Alice", 500)

# --- 2. Copying the Reference ---
# 'acc2' receives a copy of the memory address pointing to the same BankAccount instance.
acc2 = acc1

# Both variables reference the same underlying object:
print(acc1 is acc2)  # Output: True

# --- 3. Modifying State Through One Reference ---
# Depositing $200 via acc2 updates the underlying object in memory.
acc2.deposit(200)

# Reading the balance via acc1 shows $700 because both variables point to the exact same bank account.
print(acc1.balance)  # Output: 700
print(acc2.balance)  # Output: 700
```

In the example above, there is only one object, which is created by calling the constructor. However, there are two object references that refer to the same object.

If you want acc2 to be an independent account rather than an alias, you must instantiate a new object or create a copy rather than assigning acc2 = acc1.

●.7 Explain and apply the concepts of encapsulation and information hiding in OOP

Syllabus point: B3.1.5

Encapsulation means wrapping data and the methods that act on that data into one class. Information hiding means limiting outside access to internal details so that objects can protect their own state.

Encapsulation versus Information Hiding

Encapsulation

- Mechanism in OOP languages (Java, C++, Python with properties).
- Groups data and methods into a single unit (class).
- Uses access modifiers (private, protected, public) to control access.
- Provides a controlled interface for interacting with internal state.
- Example: A BankAccount class with deposit() and withdraw() methods.

Information Hiding

- Design principle (conceptual, not language-specific).
- Focuses on concealing implementation details from external code.
- Ensures users only see what's necessary (the interface), not the "secrets."
- Promotes low coupling and maintainability — internal changes don't break external code.
- Example: Users call withdraw() without knowing how balance or interest is calculated.

There are several benefits of using encapsulation in programming.

- Firstly, it promotes maintenance since any changes made to the data or the way it is stored and accessed internally can be accommodated in just one class. This means that other classes using that data can continue to use the public accessor methods and do not need to be modified.
- Secondly, encapsulation enhances data hiding and security by allowing a class to hide the way data is stored, preventing accidental access or modification. Additionally, a class can restrict the ways its data and actions are altered or called, providing an extra layer of security.
- Thirdly, encapsulation can make testing easier since all the data structure is contained in a single class. Encapsulation makes testing easier because each class manages its own data, allowing classes to be tested independently without relying on the internal details of other classes.
- Lastly, encapsulation can speed up development by allowing different programmers to work independently on different classes. Since the internal workings of each class are hidden, developers can work on multiple classes simultaneously without interfering with each other's work.

Encapsulation is implemented with:

- Public methods
- Controls of visibility of operations methods and fields.
- Data is private / not accessible

●7.1 How Python usually handles information hiding

- A leading underscore, such as `_balance`, means the attribute is intended for internal use.
- Properties, created with `@property`, allow a class to expose a value safely while still validating or controlling access.
- Methods such as `deposit()` and `withdraw()` give the class a controlled way to change its own state.

●7.2 Python example

Python

```python
class BankAccount:
    def __init__(self, owner: str, balance: float):
        self.owner = owner
        self._balance = balance

    @property
    def balance(self) -> float:
        return self._balance

    def deposit(self, amount: float) -> None:
        if amount > 0:
            self._balance += amount

    def withdraw(self, amount: float) -> bool:
        if 0 < amount <= self._balance:
            self._balance -= amount
            return True
        return False
```

The account object protects its own state. Outside code can ask for the balance, but it should not directly edit `_balance`. Instead, it must call the class methods, which enforce the rules for valid deposits and withdrawals.